Strata — Child Guide

For document authors

strata-kb v1.34.5 · 2026-11-05

1. Who this guide is for

You author documents. You bring a source PDF in, check what the splitter made of it, get it summarized, validate it, sign it off and publish it to the hub, where everyone else can search it. A repository that does this is a **child**.

> **Nothing you author is searchable until a pull request on the hub merges.** Your local `.kb/` is a drafting desk. The hub is the published world.

A child repo does	A child repo does not
Ingest source documents and keep the four layers (L0 to L3)	Host the company-wide search service, MCP server or web UI: the hub does
Summarize, validate, approve and publish them	Approve anything by itself: approval is a human act and is never automatic
Read the hub's published knowledge (`kb query`, MCP)	Publish tickets or code knowledge: `ba` and `dev` repos have their own guides

A hub repo can author documents with the same steps up to publish (section 3), so the hub guide links here instead of repeating them and lists the differences.

Shared concepts, such as the L0 to L3 layers and `kb-context` citations, are explained in the architecture document.

Slash commands. Every slash command in this guide exists for Claude Code, GitHub Copilot and Cursor, with these exceptions:

- `/kb-summarize` has no Copilot prompt. Copilot loads `.github/instructions/kb-summarize.instructions.md` while you work in `.kb/**`. Cursor also has a rule, `.cursor/rules/kb-summarize.mdc`.
- `/kb-ingest` and `/kb-publish` are skills in Claude Code (`.claude/skills/<name>/`), not files in `.claude/commands/`. You still type the slash command.
- `/kb-l3-fix` is useful only while the L3 AI fix is switched on (section 3.3).
- A child repo has no `/kb-code-sync` and no `/kb-auto`.

The commands a child gets: `/kb-init`, `/kb-docker-setup`, `/kb-mcp-setup`, `/kb-ingest`, `/kb-summarize`, `/kb-l3-fix`, `/kb-approve`, `/kb-publish`.

2. First-time setup

2.1 Install and scaffold

```
pip install "strata-kb[ingest]"
mkdir my-kb && cd my-kb && git init
kb init --kind child
```

You need Python 3.11 or newer and Git. The `ingest` extra brings the PDF pipeline; without it every other command still works, which suits a machine that only queries or publishes. No local Python for ingestion? Use Docker instead (section 2.3).

`kb init --kind child` writes `.kb/config.yaml`, `.kb/index.yaml`, `source/.gitignore`, `docker-compose.yml`, `.gitattributes`, the MCP wiring (`.mcp.json`, `.cursor/mcp.json`), the CI workflow `.github/workflows/kb-publish.yml`, `QUICKSTART.md` and the eight skills and commands for all three assistants. Re-running it is safe: it creates missing files and refreshes scaffold files, but skips `.kb/config.yaml` and `.kb/index.yaml` unless you pass `--force` (see the appendix).

2.2 Point at the hub and register the repo

```
# .kb/config.yaml
kind: child
repo_id: safety
hub: https://github.com/acme/kb-hub.git
intake: https://kb-intake.acme.com   # optional: publish from CI
```

- **`hub`** is the git URL or path of the hub. `kb query`, `kb get` and `kb tags` read it through a cached clone, and `kb publish` without `intake:` publishes into it.
- **`intake`** is the hub's intake service. Set it to publish from your own CI (section 4.2): no secret lives in this repo.
- **`repo_id`** is pre-filled from the folder name and becomes the prefix of every citation this repo produces (`safety:handbook §4.2`), so keep it short and durable; changing it later orphans the old entry on the hub. It must **exactly match** the id this repo is registered under in the hub's `federation/registry.yaml`. Ask the hub maintainer to add it. Once that file is non-empty the hub is *governed*: `kb publish` over git refuses any repo-id not listed there and refuses direct pushes, so every child must be registered, not only those that use `intake:`.

Never put a token in `hub:` or `intake:`. The file is committed, so the token lands in history. `kb doctor` warns; rewrite the URL, pass the credentialed URL in the `STRATA_KB_HUB` environment variable (it overrides the file), and rotate the token. Any machine that uses a credentialed hub URL needs **Git 2.31 or newer**: older Git ignores the mechanism that keeps the token out of the clone's config, and every hub command fails with an authentication error. CI runners are fine.

`hub:` and `STRATA_KB_HUB_URL` are different values: `hub:` is the git URL or path of the hub repo; `STRATA_KB_HUB_URL` is the hub's HTTP base URL, used by MCP.

2.3 Source files and the ingest image

Source PDFs are usually licensed or confidential. Put them in `source/`, which `kb init` told Git to ignore (`sources/` is ignored too). **Do not commit them.** The knowledge base carries the extracted text under your own control; the source stays where it legally belongs. `kb doctor` warns when git already tracks a file in either folder.

For ingestion without a local Python install, pull the ingest image:

```
kb docker-setup        # in your assistant: /kb-docker-setup
```

It checks Docker and pulls the Strata image, so ingest runs inside the container (section 4.3). Pass `--no-docker` to only print the commands. Skip it if you installed the `ingest` extra.

2.4 Connect your assistant

`.mcp.json` (Claude Code) and `.cursor/mcp.json` (Cursor) point at the hub through two placeholders, `${STRATA_KB_HUB_URL}` and `${STRATA_KB_HTTP_TOKEN}`. Run `kb mcp-setup` (in your assistant: `/kb-mcp-setup`). It asks for the hub's HTTP base URL and token, writes both into `.env`, makes sure git ignores that file, and verifies them against the hub, so a wrong URL and a rejected token come back as different errors. Get the token from the hub maintainer.

Prefer the hidden prompt or the `STRATA_KB_HTTP_TOKEN` variable over `--token`. That flag leaves the token in your shell history and in the process listing that other users on the machine can read. `kb mcp-setup` also warns when the URL is plain `http://` to a non-loopback host, because the token then crosses the network in clear.

The MCP config reads those variables from your environment, not from `.env`, at the moment your assistant starts. Load the file (`set -a; source .env; set +a`, or use direnv) and restart the assistant.

One version for every repo. Every repo runs the hub's strata-kb version. Upgrade the hub first. Then, once `kb mcp-setup` has run (above), run `kb doctor` in this repo: it installs the hub's version, sets the `strata-kb==<hub version>` pins in `.github/workflows/`, and refreshes the skills and commands. In CI it never installs, re-pins or refreshes skills; a CI job that runs `kb doctor` with `STRATA_KB_HUB_URL` and `STRATA_KB_HTTP_TOKEN` set reports a version mismatch as an error instead (none of the scaffolded workflows runs it). With `STRATA_KB_HUB` or `--hub` set, `kb doctor` skips the MCP path and the version check, and only reports that the version was not checked against the hub. The files under `.github/instructions/` and `.cursor/rules/` (the Copilot and Cursor `kb-summarize` files) refresh only through `kb init`. The pin matters: an unpinned `pip install strata-kb` in a job that holds `id-token: write` would let every run pick up whatever PyPI serves, in a job able to write to the hub.

2.5 Ask whether the hub auto-merges your repo

A hub admin can switch **auto-merge** on per repo in the hub's Registry page. Then the hub merges your publish PR right after the intake opens it, with no review on the hub side, and `kb publish` prints `kb publish: merged on the hub — <PR link>` instead of `kb publish: PR on the hub — <PR link>`. Ask the hub maintainer which you have: with auto-merge off, a person on the hub reviews and merges every publish PR.

---

3. Daily work

The authoring flow, one document at a time:

Task	Command	Result
Bring a PDF in	`kb ingest source/<file>.pdf --id <id>` or `/kb-ingest`	`.kb/<id>/` with L3 and a pending L1/L2 scaffold; the L3 check runs
Review flagged L3	`kb l3 report <id>`, then `kb l3 confirm <id> --section <s>`	the document may move on
Fix L3 structure with AI (optional)	`kb l3 fix on`, `/kb-l3-fix <id>`	drafts that a machine checks against the PDF
Summarize	automatic at ingest, else `kb summarize` or `/kb-summarize`	L1/L2 filled in
Validate	`kb build`	`kb build: OK`
Sign off	commit, then `kb approve <id>` or `/kb-approve`	sections become `reviewed`
Publish	`kb publish` or `/kb-publish`	a PR on the hub (or a merge, if auto-merge is on)

3.1 Ingest a document

```
kb ingest source/employee-handbook.pdf \
 --id hr-handbook \
 --tags hr,policy,benefits \
 --revision "2026 edition"
```

In your assistant, `/kb-ingest <file>` asks for the id, tags and revision first and then runs the command; if Docker and `docker-compose.yml` are available it uses the container (section 4.3).

Flag	What it does	Advice
`--id`	The document's permanent id	Short, lowercase, hyphenated. It appears in every citation forever.
`--tags`	Labels that pre-filter search	2 to 5. Think how a reader narrows a search.
`--revision`	Edition label	Always set it: it appears in every citation.
`--sections`	Only these chapters, e.g. `5,6`	A trial run on a long document, or re-ingesting one chapter.
`--chapter-pattern`, `--appendix-pattern`, `--attachment-pattern`	Heading regexes	The defaults are `Chapter N`, `Appendix X`, `Attachment N`; the last ones you used are remembered.
`--no-bookmarks`	Split by heading pattern, not the PDF outline	Only if the outline is missing or wrong.
`--no-summarize`	Skip the automatic summarize step	You want to inspect the split first.
`--llm claude|copilot|none`	Pick the summarize runner	Default: auto-detect.
`--no-llm-review`	Skip the agent's verdicts on L3 findings	Section 3.2.

Ingest writes:

```
.kb/hr-handbook/
├── _manifest.yaml                L1: every section, all "pending"
├── ch4-benefits.md               L2: tables in place, prose blank
├── ch4-benefits.raw.md           L3: the full original
├── _l3check.yaml                 the L3 check (section 3.2)
└── assets/<sha256>.webp          figures, stored once each
```

It also keeps a scratch cache in `.kb-work/`; keep that folder out of git. The first run on a machine downloads a page-layout model (about 500 MB) and caches it.

Read the ingest report. It names every judgement call the splitter made: headings demoted to text, ids it had to invent, numbered headings out of order, duplicate ids it renamed, bookmarks it could not match. Many fallback ids or out-of-order headings usually mean the outline is wrong: try `--no-bookmarks` with a heading pattern.

Re-ingest. Without `--sections`, ingest is a full replace: every Markdown file of the document is deleted and rewritten, including summaries you already reviewed. With `--sections 6`, only that chapter's files and manifest entries are rewritten; the rest, reviewed summaries included, is untouched. A `--sections` value that matches no heading **deletes** that chapter, and the report says so: read it.

Ingest a Markdown file. A `.md` or `.markdown` file goes straight to L3, with no ingest extra and no PDF:

```
kb ingest source/policy.md --id policy
```

Only heading lines are rewritten, by the same id rules as a PDF (a numbered heading keeps its number, an unnumbered one gets a slug id under its parent, and an unnumbered heading nests by its `#` level); every other byte of the file reaches L3 unchanged. Local `.png` and `.webp` images inside the source folder are copied to `assets/<sha256>.<ext>` and their links rewritten; any other image stays as written, with a warning. Ingest refuses a file whose fenced code or HTML block holds a `##` or `###` line (it lists the line numbers: move the line out or indent it), and an id already used for the other kind: `'<id>' was ingested from a PDF — ingest the Markdown under a new --id`, and the reverse. `kb l3 fix apply`, `kb l3 lint` and `kb l3 fix pages` take `--source <file>` (`--pdf` still works); a Markdown document has no page images, so `kb l3 fix pages` and `--from-image` refuse with `source is Markdown: no page images`. The source Markdown is the master: to change the text, edit it and ingest again. That resets the AI-fix record, so run `/kb-l3-fix` and confirm again. `kb authored <doc>` still freezes a document.

3.2 Check L3 against the PDF

After writing L3, `kb ingest` compares every section with the PDF's own text layer and writes the findings to `.kb/<id>/_l3check.yaml` (commit it; it is never sent to the hub). It flags a section when its id is a fallback, it is out of order, text is missing, extra or garbled, a paragraph is broken in the middle, or the pages have no text layer to check. An agent (the `claude` or `copilot` CLI it finds, with the `llm.model` from `.kb/index.yaml`) adds a verdict to each broken-paragraph finding: `spurious`, `intentional` or `unsure`. It never clears a flag and never edits L3.

A flagged document cannot move on. `kb summarize` skips it (exit 1), `kb build` errors, and `kb publish` and `kb ci-publish` refuse. Ingest then prints `Summarize skipped until the L3 review is confirmed`. Nothing flagged means nothing to do.

```
kb l3 report hr-handbook                        # findings, evidence, verdicts (exit 1 while any)
kb l3 confirm hr-handbook --section 4.12        # a reviewer checked this one
kb l3 confirm hr-handbook --all --yes           # every flagged section
```

Open the PDF page the report names and compare. Confirm only what you checked; `--by 'name <email>'` overrides the git identity. A confirmation holds only while that section's L3 is unchanged; re-ingesting a changed section asks again.

3.3 Fix the structure of L3 with AI (optional)

Ingest can leave structure problems in L3: a fallback id, a wrong heading level, a paragraph broken in the middle, a repeated block, a page header or footer in the text. The L3 AI fix lets an assistant repair that structure while a machine checks it changed nothing else. It is off by default (`l3_fix.enabled` in `.kb/config.yaml`), and L3 stays the PDF's text.

```
kb l3 fix on                                    # also adds .kb-work/ to .gitignore
kb ingest source/handbook.pdf --id hr-handbook # records a baseline; does not summarize
kb l3 lint hr-handbook --pdf source/handbook.pdf
# run /kb-l3-fix hr-handbook in your assistant: it writes drafts under .kb-work/
kb l3 fix apply hr-handbook --pdf source/handbook.pdf --assistant claude-code
kb l3 report hr-handbook                        # AI fixes first, each with a git diff hint
```

```
kb l3 confirm hr-handbook --section 4.12       # a human checked it
kb summarize
```

- `--assistant` is `claude-code`, `copilot`, `cursor` or `other`: the tool that wrote the drafts. `kb l3 lint` is advisory (layout rules R1 to R9, exit 1 on any issue).
- `kb l3 fix apply` checks each draft against the PDF and writes nothing for a draft that fails. Each line reads `§<name> applied → <new ids>`, `§<name> unchanged` or `§<name> rejected: <check>: <detail>`; it exits 1 on any rejection. The checks (tables byte for byte, every word against the PDF's text layer) are described in the skill.
- For a scanned section or a broken font, `kb l3 fix pages <id> --pdf <pdf> --page N` renders PNG pages to `.kb-work/<id>/pages/` for the assistant, and `apply --from-image <name>` skips the word check. On a section whose text layer is usable it is allowed only for heading lines (a heading with a broken font); the rest of the draft is still checked word by word. That fix is marked unverified and listed first by `kb l3 report`: read it against the PDF.
- While the switch is on, ingest does not summarize. While an AI fix awaits review, or L3 was edited outside `apply`, the same gate as 3.2 holds. Commit `_l3fix.yaml`; it is never sent to the hub. `kb l3 fix off` turns the switch off.

The AI never confirms. The machine checks tables and words, not whether a heading level or a merged paragraph is what the author meant. You read the `git diff` against the PDF and confirm.

3.4 Summarize

This is the only step a model writes prose in. `kb ingest` runs it automatically when it finds the `claude` or `copilot` CLI (config: `llm:` in `.kb/index.yaml`; `--llm` pins one). Otherwise sections stay `pending`:

```
kb status                   # sections still pending summarization
kb summarize hr-handbook    # run or retry
```

With no LLM CLI, run `/kb-summarize` in your assistant instead: it reads `kb status`, fans the pending sections out to read-only sub-agents and merges the result under the same rules. The rules (the summary keeps the document's language and never rephrases codes, numbers, units or cross-references; tables are never touched) live in the skill. A section that fails stays pending.

To redo summaries:

```
kb summarize hr-handbook --redo --section 4.12       # one section
kb summarize hr-handbook --redo --dry-run            # see the plan first
kb summarize --redo --all                            # every document
kb summarize hr-handbook --print-prompt --section 4.12
```

`--redo` skips sections a human already reviewed unless you add `--include-reviewed --yes`.

3.5 Validate with `kb build`

```
kb build
```

```
kb build: OK
```

`kb build` fails on a leftover `TODO` marker or empty summary, on any table in L2 that does not match the L3 original (in both directions, order included; this check cannot be waived), on L3 changed after its summary was written, on L2

changed after approval, and on an unconfirmed L3 finding. Quality rules (summary length, vocabulary overlap with the original, invented codes) are warnings; `--strict` makes them errors. It writes the manifest's token counts only when it reports no error.

```
kb build --allow-pending   # validate finished work while the rest is in progress
kb build --strict          # what kb approve requires
```

On a table mismatch, copy the table verbatim from the section's `.raw.md` (L3) over the one in the `.md` (L2). Do not retype or reformat it.

3.6 Approve

Check the summaries before you sign them off. Read the `.md` files, not `.raw.md`.

- Does the prose match your understanding of the subject? Is anything important omitted, or present that the source does not say?
- Is every code, identifier, number and unit exactly as in the source?
- Tables: the build compared them, but glance for extraction errors (a row or column the PDF reader misread); a machine cannot catch that.
- Do the one-line summaries in `_manifest.yaml` say what each section is about, well enough to be found by search?

`kb approve` signs the **committed** content, so commit first (`kb build` rewrites the manifest's token counts, so build, then commit, then approve):

```
git add .kb/hr-handbook && git commit -m "kb: hr-handbook summaries"
kb approve hr-handbook                       # in your assistant: /kb-approve
kb approve hr-handbook --section 5.3         # only some sections (repeatable)
kb approve hr-handbook --by "Jane Smith <jane@acme.com>"
git add .kb/hr-handbook && git commit -m "kb: approve hr-handbook"
```

It refuses a dirty `.kb/<id>` tree and a failing `kb build --strict`, flips `summarized` sections to `reviewed`, and records `reviewed: {by, at, l2_sha256}` for each (`--by` defaults to your git identity), so a later edit to an approved summary is detected. `kb approve --all-changed --against <rev>` approves what changed since a revision. **Approval is a human act and is never automatic**: the assistant runs it only when you ask.

3.7 Publish

Preview, then publish:

```
kb diff hr-handbook --against HEAD   # added, removed and changed sections
kb publish                           # in your assistant: /kb-publish (diff, confirm, publish)
```

`/kb-publish` shows the diff, asks for one go or no-go, then runs `kb publish`. The command first prints `kind: kb → hub menu: Document`, then:

Your config	What `kb publish` does
`intake:` set	Needs a clean, committed `.kb/`. Tags the commit `kb-publish/<ts>`, pushes the tag, and your CI publishes (section 4.2). The command waits up to 600 seconds and prints the PR link.
`hub:` only	Mirrors `.kb/` into `federation/<repo-id>/` on the hub and opens a PR there. It needs `gh` able to open a PR on the hub's host. A hub path on disk with no git remote gets a direct commit instead.

`--pr` and `--direct` force a mode: `--direct` commits straight to the hub and is refused on a governed hub that has a git remote; with neither flag `kb publish` picks direct only for a hub with no git remote, and otherwise a PR or a clear refusal. `kb publish` has no `--intake` flag: it reads `intake:` from the config. `--hub` or the `STRATA_KB_HUB` variable overrides `hub:`.

Publishing copies `.kb/` **artefacts only**: documents, indexes, manifests and `assets/`. Configuration files and dotfiles stay behind, and any withheld file is named in a `[warn] ... (allowlist)` line. `_l3check.yaml` and `_l3fix.yaml` are also withheld, but silently. That is deliberate: a `config.yaml` that someone put a credential in must never reach the hub.

`kb publish` and `kb ci-publish` warn how many sections ship unreviewed; `--require-reviewed` makes that fatal. An unconfirmed L3 finding always blocks. Content becomes searchable when the PR merges on the hub (or at once, with auto-merge).

3.8 Read the hub

```
kb query "parental leave eligibility"
kb query "parental leave" --tags hr --budget 600
kb get hr-handbook 4.12 --level l2     # or --level l3
kb tags                                # every tag published on the hub
```

Every result starts with `--- [<repo-id>:<doc-id> §<section> (<revision>)] match=<mode> ~<n>tk` (`keyword`, `semantic` or `hybrid`) and cites that same form. Query in the document's own language: the store never translates. `--budget` is a token ceiling and is advisory: the first result is always returned whatever its size, because a section is never cut. `--tags` also accepts a document id; `--kind kb|ba|dev` filters by hub menu. The same search is available to your assistant over MCP (section 2.4).

---

4. Automation modes

A child has no `kb auto`. Four things run without you; none of them approves or confirms.

Mode	What it does	Switch
Auto-summarize	`kb ingest` summarizes with the `claude` or `copilot` CLI it finds	on when a CLI is found; `--no-summarize`, `--llm none`, or `llm.runner: none` in `.kb/index.yaml`
L3 AI fix	an assistant repairs L3 structure, a machine checks it (3.3)	`kb l3 fix on` / `off`
CI publish via OIDC	your own GitHub Actions job publishes the tagged commit	`intake:` in `.kb/config.yaml`
Docker ingest	ingest runs in a container, no local Python	`kb docker-setup`

`.kb/index.yaml llm:` holds `runner` (`auto`, `claude`, `copilot`, `none`), `model`, `effort`, `max_workers` and `timeout`. `--llm` on `kb ingest` or `kb summarize` overrides the runner for one run.

4.1 What always stays human

Approval is never automatic (3.6), the AI never confirms an L3 finding or an AI fix (3.2, 3.3), and publishing is yours to start. Auto-summarized sections are `summarized`, not `reviewed`, until you run `kb approve`.

4.2 CI publish via OIDC

`kb ci-publish` is the CI route: it runs in this repo's own GitHub Actions job, authenticates to the hub's intake service with GitHub Actions OIDC, and needs no secrets. The scaffolded `.github/workflows/kb-publish.yml` does it:

1. `kb publish` (with `intake:` set) tags the committed `.kb/` as `kb-publish/<ts>` and pushes the tag.
2. The workflow, triggered by `kb-publish/*` tags, installs `strata-kb` (pinned to the hub's version, section 2.4) and runs `kb ci-publish` with `id-token: write`.
3. The intake opens the PR on the hub (and merges it, if the hub's Registry has auto-merge on for your repo). `kb publish` polls for the PR link and times out after 600 seconds.

The intake accepts only `refs/tags/kb-publish/*`. In a child repo that tag is the only trigger: **merging to the default branch publishes nothing.** The registry entry for your repo must leave `workflow:` unset, because the pin is compared with a reference that ends in the tag name and never matches. The hub maintainer registers `owner/repo` with your `repo_id`; if `repo_id` differs, the workflow still succeeds and a PR opens, but `kb publish` times out waiting for a PR it cannot find, and uploads lose incrementality (every publish becomes a full upload).

`kb ci-publish` also takes `--intake`/`STRATA_KB_INTAKE` and `--require-reviewed`; the workflow runs it bare, so set `intake:` in the config instead.

4.3 Docker ingest

`docker-compose.yml` defines one service, `hub`, that runs a one-shot ingest in the Strata image: the repo is mounted at `/data` and the layout model is cached in a named volume.

```
docker compose run --rm hub kb ingest source/handbook.pdf --id hr-handbook --no-summarize
```

The container has no LLM CLI, which is why `--no-summarize` is explicit. Summarize afterwards on your machine (`kb summarize`) or with `/kb-summarize`. `/kb-ingest` and `/kb-l3-fix` pick the container on their own when `docker info` succeeds and `docker-compose.yml` exists, and prefix their `kb` commands with `docker compose run --rm hub`.

---

5. Troubleshooting



6. Appendix

6.1 Config keys and environment variables

Key or variable	Meaning
`kind`, `hub`, `repo_id`, `intake`	see 2.2. Never put a token in `hub:` or `intake:`
`l3_fix.enabled`	the L3 AI fix switch, default `false`; set by `kb l3 fix on` / `off`
`.kb/index.yaml` `llm:`	`runner` (`auto`, `claude`, `copilot`, `none`), `model`, `effort`, `max_workers`, `timeout`
`STRATA_KB_HUB`	overrides `hub:`; carries a credentialed URL in CI. Also read by `kb publish --hub` and `kb doctor --hub`
`STRATA_KB_HUB_URL`, `STRATA_KB_HTTP_TOKEN`	the hub's HTTP base URL and bearer token: MCP and `kb doctor` over MCP
`STRATA_KB_INTAKE`	same as `kb ci-publish --intake`
`STRATA_KB_HUB_CACHE`, `STRATA_KB_HUB_TTL`	the local hub clone `kb query` reads: its folder (default `~/.strata-kb/hub/`) and how long it counts as fresh (default 900 seconds)

`asset_store` is hub-only: a child's figures travel with the publish.

Two reminders. `kb mcp-setup --token` leaves the token in your shell history: prefer the hidden prompt or `STRATA_KB_HTTP_TOKEN`. `kb publish` and `kb ci-publish` warn how many sections ship unreviewed; `--require-reviewed` makes that fatal.

6.2 What is committed, and what is not

Path	Holds	Committed
`.kb/<id>/` (manifest, `.md`, `.raw.md`, `assets/`)	the document	yes
`.kb/<id>/_l3check.yaml`, `_l3fix.yaml`	L3 check state and AI-fix record	yes, but never published
`source/`, `sources/`	source PDFs	no, never
`.kb-work/`	ingest scratch, AI-fix drafts, page images	no
`.env`	MCP URL and token	no (`kb mcp-setup` ignores it)

6.3 Rules the L3 AI fix skill follows

The `kb-l3-fix` skill runs only while the switch is on; it writes drafts and never edits `.kb/`; it never runs `kb l3 confirm`; it keeps the PDF's text (no added, reworded, translated or summarized text) and every table byte for byte; and it retries a rejected draft at most twice before leaving the section to you. Details and the check order are in the skill itself.

6.4 Exit codes

`0` ok, `1` error or refusal (a `kb build` error, `kb l3 report` while findings are listed, `kb l3 fix apply` with a rejection), `2` a usage error, a stale hub cache in `kb doctor`, or a stale citation in `kb resolve`.

6.5 Re-running `kb init`, Windows, and the hub's side

`kb init` skips `.kb/config.yaml` and `.kb/index.yaml` on a re-run; `--force` replaces them. A scaffold file named `*.local.md` is never refreshed, even with `--force`. `kb doctor` refreshes only the agent files (`.claude/skills/`, `.claude/commands/`, `.github/prompts/`, `.cursor/commands/`).

`kb` forces UTF-8 on its streams. On Windows, long document paths need `git config core.longpaths true`. `.gitattributes` keeps `.kb/**` untouched by line-ending conversion (`-text`).

The hub guide covers what the hub operator does with your publish: registering the repo, auto-merge, and reviewing the PR.